pom.xml

```
<project xmlns="http://apache.org"
         xmlns:xsi="http://w3.org"
         xsi:schemaLocation="http://apache.org https://apache.org">
<modelVersion>4.0.0</modelVersion>
<groupId>com.demo</groupId>
<artifactId>exam-app</artifactId>
<version>1.0</version>
 <properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
</properties>
<dependencies>
  <!-- Core Struts 2 -->
  <dependency>
     <groupId>org.apache.struts</groupId>
     <artifactId>struts2-core</artifactId>
     <version>6.3.0</version>
  </dependency>
  <!-- CRITICAL FIX: The missing official RabbitMQ Java Client Driver -->
  <dependency>
     <groupId>com.rabbitmq</groupId>
     <artifactId>amqp-client</artifactId>
     <version>5.22.0</version>
  </dependency>
  <!-- In-Memory Zero-Setup RabbitMQ Broker Simulator -->
  <dependency>
     <groupId>com.github.fridujo</groupId>
     <artifactId>rabbitmq-mock</artifactId>
     <version>1.1.1</version>
```

```
  </dependency>
</dependencies>
</project>
```

ExamRunner.java

```
package com.demo;
import com.github.fridujo.rabbitmq.mock.MockConnectionFactory;
import com.rabbitmq.client.Channel;
import com.rabbitmq.client.Connection;
import com.rabbitmq.client.ConnectionFactory;
import com.rabbitmq.client.GetResponse;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import java.awt.*;
// ============================================================
// 1. THE STRUTS ACTION CLASS (Holds data and queue logic)
// ============================================================
class ParentAction {
  private String studentName;
  private String fatherName;
  private String motherName;
  private String displayMessage;
  private static final ConnectionFactory factory = new MockConnectionFactory();
  private static final String QUEUE_NAME = "exam_student_buffer_queue";
  public String submitAndRetrieve() throws Exception {
     // PRODUCER: Store into the RAM memory buffer
     try (Connection conn = factory.newConnection(); Channel ch = conn.createChannel()) {
        ch.queueDeclare(QUEUE_NAME, false, false, false, null);
        String payload = "Student: " + studentName + " | Father: " + fatherName + " | Mother: " +
motherName;
```

```
            ch.basicPublish("", QUEUE_NAME, null, payload.getBytes("UTF-8"));
        }
        // CONSUMER: Retrieve back from the RAM memory buffer
        try (Connection conn = factory.newConnection(); Channel ch = conn.createChannel()) {
            ch.queueDeclare(QUEUE_NAME, false, false, false, null);
            GetResponse response = ch.basicGet(QUEUE_NAME, true);

            if (response != null) {
                displayMessage = "Success! Data processed via RabbitMQ Buffer:\n" + new
String(response.getBody(), "UTF-8");
            } else {
                displayMessage = "The buffer queue is empty.";
            }
        }
        return "success";
    }
    public void setStudentName(String s) { this.studentName = s; }
    public void setFatherName(String f) { this.fatherName = f; }
    public void setMotherName(String m) { this.motherName = m; }
    public String getDisplayMessage() { return displayMessage; }
}
// ============================================================
// 2. THE SYSTEM EXECUTOR (Locked Layout Application Window)
// ============================================================
public class ExamRunner {
    public static void main(String[] args) {
        ParentAction actionInstance = new ParentAction();
        // 1. Create the application frame window (Uses Centering Engine)
        JFrame frame = new JFrame("Student Registration Form");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(450, 320);
```

```
        frame.setMinimumSize(new Dimension(400, 280));
        frame.setLayout(new GridBagLayout()); // Locks child layouts in the absolute center
        frame.setLocationRelativeTo(null);
        // 2. Create the inner form container card with a strict static dimension
        JPanel formCard = new JPanel(new GridLayout(4, 2, 10, 15));
        formCard.setPreferredSize(new Dimension(340, 180)); // Restricts layout components from
stretching
        // 3. Create input boxes and submit trigger buttons
        JTextField txtStudent = new JTextField();
        JTextField txtFather = new JTextField();
        JTextField txtMother = new JTextField();
        JButton btnSubmit = new JButton("Submit & Process");
        btnSubmit.setBackground(new Color(41, 128, 185));
        btnSubmit.setForeground(Color.WHITE);
        btnSubmit.setFocusPainted(false);
        // 4. Populate layout matrix cards
        formCard.add(new JLabel("Student Name:")); formCard.add(txtStudent);
        formCard.add(new JLabel("Father Name:"));  formCard.add(txtFather);
        formCard.add(new JLabel("Mother Name:"));  formCard.add(txtMother);
        formCard.add(new JLabel(""));              formCard.add(btnSubmit);
        // Append the locked card directly inside the center framework structure
        frame.add(formCard);
        // 5. Wire submission trigger interface click mechanics
        btnSubmit.addActionListener(e -> {
            try {
                actionInstance.setStudentName(txtStudent.getText());
                actionInstance.setFatherName(txtFather.getText());
                actionInstance.setMotherName(txtMother.getText());
                actionInstance.submitAndRetrieve();
                JOptionPane.showMessageDialog(frame, actionInstance.getDisplayMessage(),
"RabbitMQ Processing Report", JOptionPane.INFORMATION_MESSAGE);
```

```
            txtStudent.setText(""); txtFather.setText(""); txtMother.setText("");
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(frame, "Error: " + ex.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
        }
    });
    frame.setVisible(true);
  }
}
```